0 1 2 3 4 5 6 7 8

X ~~8~~ ~~3~~ (5) 6 7 8 9 10 1 2 3 , target = 10

low = 0 high = 8
mid = 4

low = 5 high = 8
mid = 6

low = 5 high = 5
mid = 5

[5 5 5 5 5 2 3 4], 3

1) If arr[mid] >= arr[0]
→ left half is sorted.

otherwise
→ right half is sorted.

2) If target can be present in the sorted half, then move to the sorted half. Otherwise, discard the sorted half.

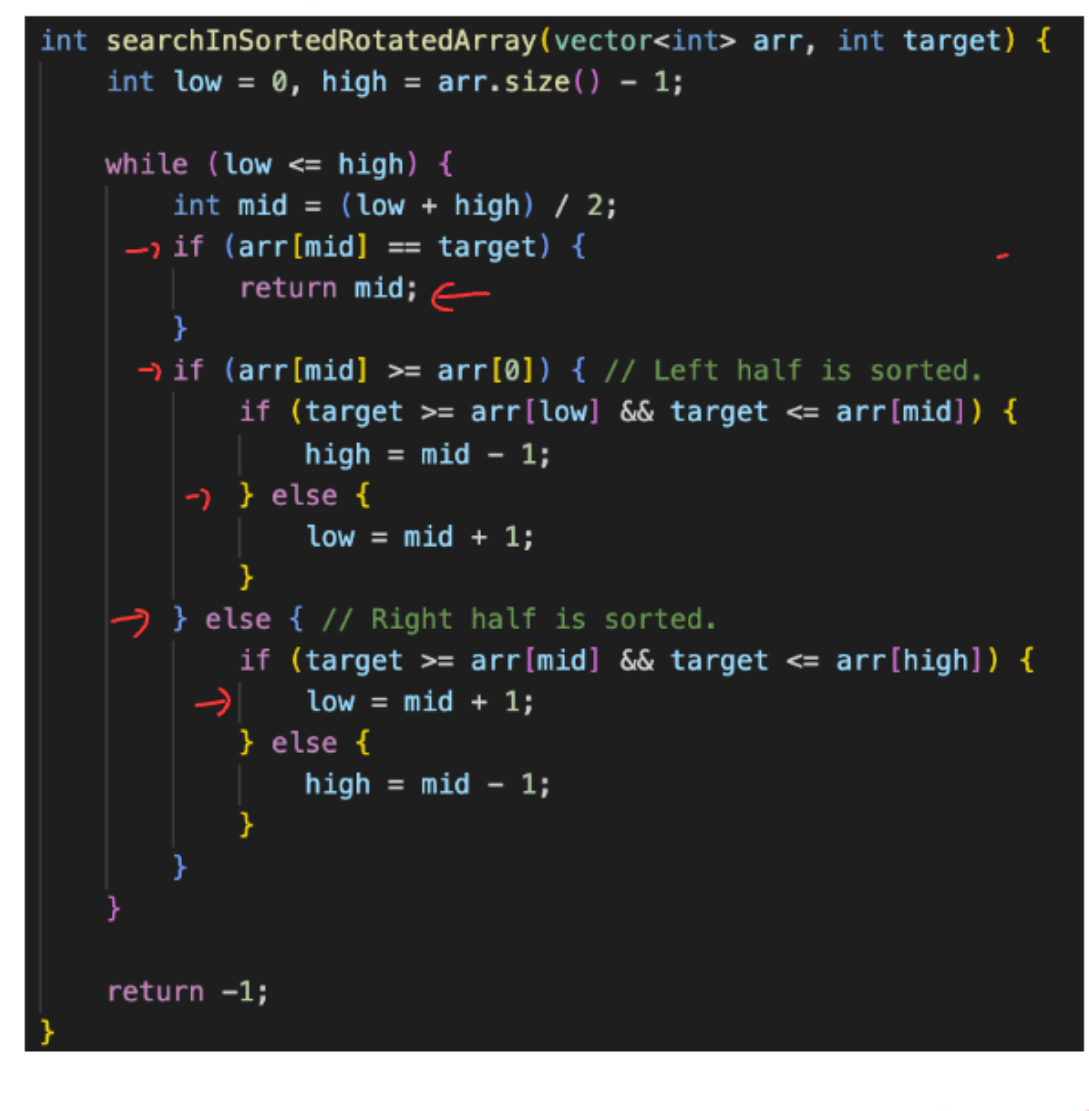
TC: O(log(n))
AS: O(1).

0 1 2 3 4 5 6 7 8
[5, 6, 7, 8, 9, 10, 1, 2, 3] target = 2

low = 0 high = 8
mid = 4

low = 5 high = 8
mid = 6

low = 7 high = 8
mid = 7



```
int searchInSortedRotatedArray(vector<int> arr, int target) {
    int low = 0, high = arr.size() - 1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) {
            return mid;
        }
        if (arr[mid] >= arr[0]) { // Left half is sorted.
            if (target >= arr[low] && target <= arr[mid]) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        } else { // Right half is sorted.
            if (target >= arr[mid] && target <= arr[high]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }

    return -1;
}
```

A = [10, 20, 10, 30]   M = 2

[10] [20, 10, 30] : (10, 60) → 60
[10, 20] [10, 30] : (30, 40) → $\boxed{40}$
[10, 20, 10] [30] : (40, 30) → 40

$\sim N^{M-1} \times N = \sim N^M$

[10 20 10 30]   35
10 30 40 70

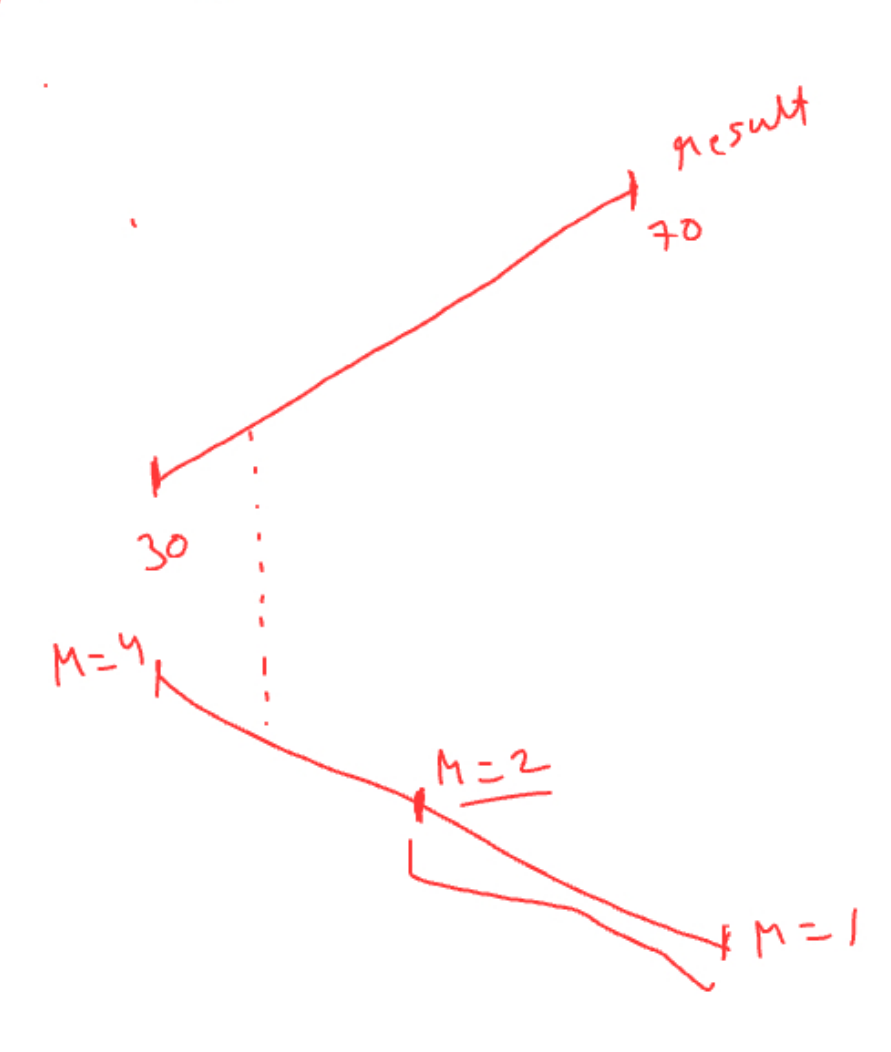
M = 3

[10 20 30 1000]   520.
~~10~~ ~~30~~ ~~40~~ 1040   340

lower bound: 90 → M = N
Upper bound: 203 → M = 1

[10, 20, 10, 30] , M = 2

low = 30 (M = 4)
high = 70 (M = 1)



low = 30 high = 70
mid = 50
[10, 20, 10, 30] 1 1 1 2
result = 50

low = 30 high = 49
mid = 79/2 = 39
[10, 20, 10, 30] 1 1 2 3

low = 40 high = 49
mid = 89/2 = 44
[10 20 10 30] 1 1 1 2
result = 44

low = 40 high = 43
mid = 83/2 = 41
[10, 20, 10, 30] 1 1 1 2
result = 41

low = 40 high = 40
mid = 40
[10 20 10 30] 1 1 1 2
$\boxed{\text{result} = 40}$

low = 40 high = 39
STOP!

TODO: Painter's Partition Problem

TC: O(n * log(sum))
AS: O(1)

[10, 20, 10, 30]
mid = 50
num Students = ~~2~~ 1
numPages = ~~0~~ ~~10~~ ~~30~~ ~~40~~ 30

---

3   30 ← ~~38~~
34 ← 43   60
37   51   69

target = 37

3   30 ← 38
34 ← 43   60
← 37   51   69

tar = 36

TC: O(n+m)
AS: O(1)

[1, 2, 3, 4]

[1, 2]   [3, 4]
2   12

x/y = 2
2xy = 1/12 /9998

arr: [1, 2, 3, 4]
PP: [1, 2, 6, (24)]
SP: [(24), 24, 12, 4]
ans: [24, 12, 8, 6]

ans[i] = PP[i-1] × SP[i+1]

PP[i] = $a_0 \times a_1 \times a_2 \cdots \times a_{i-1}$
SP[i] = $a_{i+1} \times a_{i+2} \cdots \times a_{n-1}$

arr: [1, 2, 3, 4]
PP: [1, 1, 2, 6]
SP: [24, 12, 4, 1]
ans: [24, 12, 8, 6]

ans[i] = PP[i] × SP[i]

TC: O(n)
AS: O(n)

$PP_i = \underbrace{a_0 \times a_1 \times a_2 \cdots \times a_{i-2}}_{PP_{i-1}} \times a_{i-1}$

[1, 2, 5, 6, 10], target = 4
False.

[1, 2, 2, 5, 6, 10], tar = 4
True.

BF:
i: 0 → n-1
    j: i+1 → n-1
        ai + aj == target
        return True.

TC = O(n²)
AS = O(1)

Binary Search
i: 0 → n-1
    Binary search for (tar - ai) in [ai+1 ... n-1]

TC = O(n log n)
AS = O(1)

Hashmap-Based
i: 0 → n-1
    if (tar - ai) in the hashmap
        return True.
    insert ai into the hashmap.

[1, 2, 5, 6, 10], tar = 8

1, 2, 5

TC: O(n)
AS: O(n)

Two-pointer Approach.

[1, 2, 5, 6, 10] tar = 8

11 ~~7~~ 8

TC: O(n)
AS: O(1)

TC:
O(n log n + n²) = O(n²)
AS: O(1)

i: 0 → n-1
    Two ptr to find pair with sum
    (tar - arr[i])

2 - sum → O(n)
3 → sum → O(n²)
4 → sum → O(n³)
⋮
K → sum → O(n^{K-1})

Basic case.
K == 2